# Scripting & Parallelization SLURM

## Sbatch, JobArray, OpenMP/MPI

**Daniel Valle Millares**
**(Bioinformatics Platform - CIBERINFEC)**

**BU-ISCIII**
**29 sept - 03 de octubre de 2025**
**1ª edición**



# Index

1. **Scripting on the cluster — sbatch & job arrays**
2. Parallel computing on the cluster — OpenMP vs MPI (when to use each)
3. Nextflow pipelines in HPC  - Deploy a Nextflow DSL2 pipeline in HPC
4. Wrap-up & Q&A

# Sbatch File

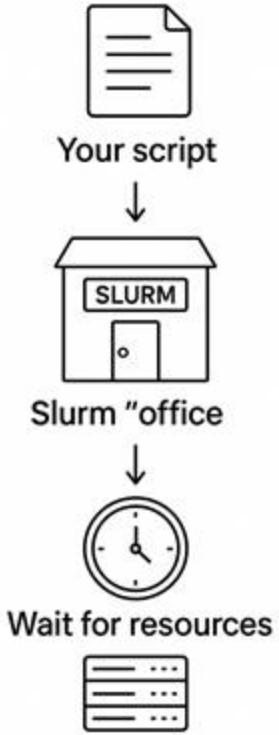
# Scripting on the cluster — Slurm: sbatch

## What is **sbatch**?

- **Command** to submit a job script to Slurm; runs in the background on compute nodes.

- Delegate the work to de cluster:
  - Submit once, let the cluster work

- Example:
  - Your script → Slurm "office" → Wait for resources → Cluster runs it



# Scripting on the cluster — Slurm: sbatch

## Set up a sbatch script file

- 0) Open a <file-name>.**sbatch** file
- 1) Init with shebang



```
#!/bin/bash
#SBATCH --option=value
#SBATCH --option=value
#SBATCH --option=value
# From here on, the commands
command_1
command_2
```

# Scripting on the cluster — Slurm: sbatch

## Set up a sbatch script file

- 0) Open a <file-name>.sbatch file
- 1) Init with shebang
- 2) Set up slurm directives at the beginning of the script by using “#SBATCH”



```
#!/bin/bash
#SBATCH --option=value
#SBATCH --option=value
#SBATCH --option=value
# From here on, the commands
command_1
command_2
```

**List of SLURM params:**
**https://slurm.schedmd.com/sbatch.html**

# Scripting on the cluster — Slurm: sbatch

## Set up a sbatch script file

- 0) Open a <file-name>.sbatch file
- 1) Init with shebang
- 2) Set up slurm directives at the beginning of the script by using “#SBATCH”
- 3) **After directives, you can:**
  - Load dependencies with ‘module load’
  - Set your commands you want to run



```
#!/bin/bash
#SBATCH --option=value
#SBATCH --option=value
#SBATCH --option=value
# From here on, the commands
command_1
command_2
```

**List of SLURM params:**
**https://slurm.schedmd.com/sbatch.html**

# Scripting on the cluster — Slurm: sbatch

```
#!/bin/bash
#SBATCH --chdir=/path/to/working/directory   # Folder where the analysis will run
#SBATCH --job-name=my_first_slurm_job        # A recognizable job name
#SBATCH --cpus-per-task=1                    # Number of CPU cores (threads) for this job
#SBATCH --mem=1G                             # RAM to reserve
#SBATCH --time=00:10:00                      # Time limit (HH:MM:SS)
#SBATCH --partition=short_idx                # Queue/partition to run in
#SBATCH --output=slurm-%j.out                # File for standard output
#SBATCH --error=slurm-%j.err                 # File for standard error
# From here on, the commands we want to run:
command_1
command_2
```

```
#!/bin/bash
#SBATCH --job-name=fastqc_demo
#SBATCH --partition=short_idx
#SBATCH --cpus-per-task=4
#SBATCH --mem=4G
#SBATCH --time=00:05:00
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
      You, 3 days ago • Update scripting and Nextflow practical ses
# Carga dependencias
module load FastQC/0.11.9-Java-11

# Crea la carpeta de resultados
mkdir -p 01-fastqc-demo-results

# Ejecuta fastqc
fastqc \
  -t 4 \ # we can also use slurm env var: "$SLURM_CPUS_PER_TASK"
  ../00-reads/virus1_R1.fastq.gz \
  ../00-reads/virus1_R2.fastq.gz \
  -o 01-fastqc-demo-results
```



Slurm ENV Vars

# Scripting on the cluster — Slurm: deploy with sbatch

Lanza el trabajo y monitoriza:

```
sbatch --chdir="$PWD" fastqc_demo.sbatch
#Submitted batch job 6417753
```

# Scripting on the cluster — Slurm: deploy with sbatch

Lanza el trabajo y monitoriza:

```
sbatch --chdir="$PWD" fastqc_demo.sbatch
#Submitted batch job 6417753
```

Slurm JobID



# Scripting on the cluster — Slurm: monitor with squeue

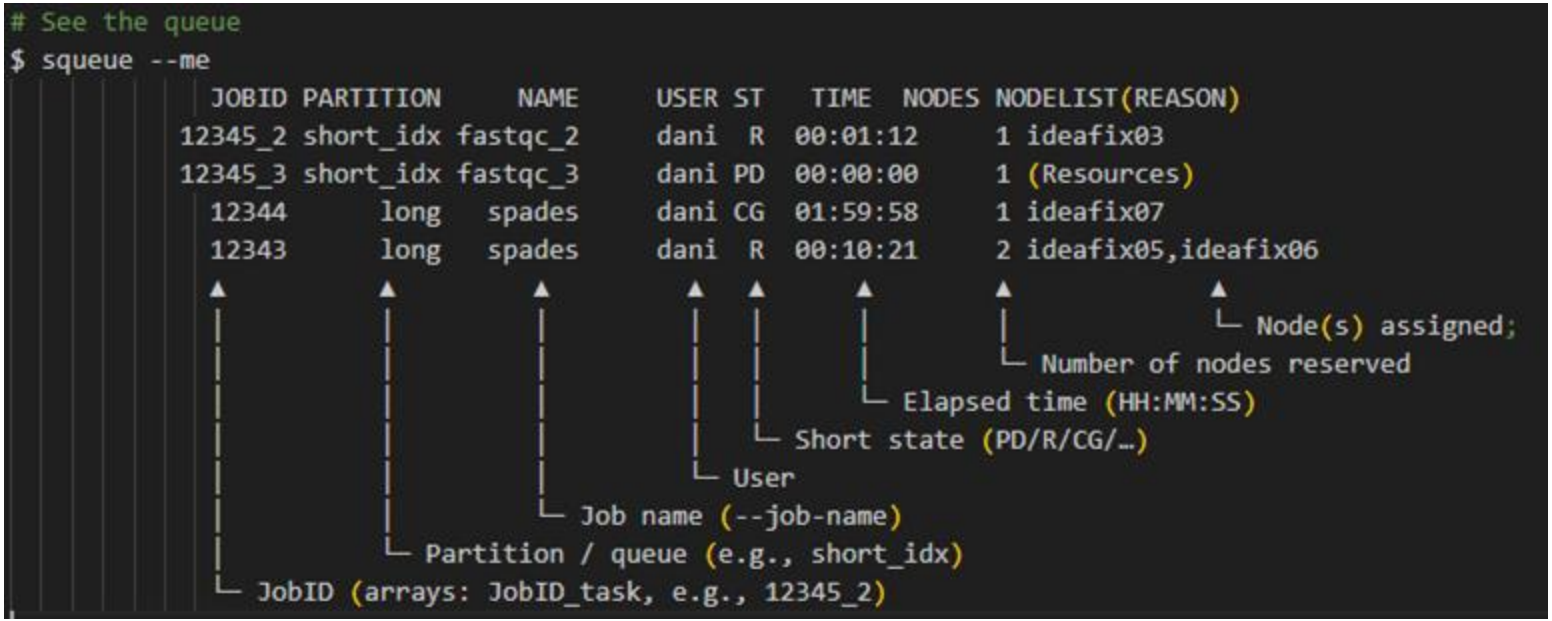
Lanza el trabajo y monitoriza:

```
sbatch --chdir="$PWD" fastqc_demo.sbatch
#Submitted batch job 6417753
```



```
squeue --me
```

# Scripting on the cluster — Slurm: monitor with squeue



# Scripting on the cluster — Slurm: monitor with scontrol

```
$: scontrol show job 6424713

JobId=6424713 JobName=bash
   UserId=mgonzalez(3088) GroupId=mgonzalez(3088) MCS_label=N/A
   Priority=70304 Nice=0 Account=ieg QOS=normal
   JobState=RUNNING Reason=None Dependency=(null)
   Requeue=1 Restarts=0 BatchFlag=0 Reboot=0 ExitCode=0:0
   RunTime=00:39:23 TimeLimit=08:00:00 TimeMin=N/A
   SubmitTime=2026-10-20T09:42:27 EligibleTime=2026-10-20T09:42:27
   AccrueTime=Unknown
   StartTime=2026-10-20T09:42:27 EndTime=2026-10-20T17:42:27 Deadline=N/A
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-10-20T09:42:27 Scheduler=Main
   Partition=short_idx AllocNode:Sid=portutatis03:2426725
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=ideafix02
   BatchHost=ideafix02
   NumNodes=1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   ReqTRES=cpu=1,mem=30G,node=1,billing=1
   AllocTRES=cpu=1,mem=30G,node=1,billing=1
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=30G MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
   Command=bash
   WorkDir=/scratch/hpc_course/da.valle
   Power=
```

# Scripting on the cluster — Slurm: monitor with sacct

```
$: sacct -j 6417753 -o JobID,JobName,State,Elapsed,MaxRSS,AllocCPUS,TotalCPU,ExitCode

JobID           JobName      State    Elapsed     MaxRSS  AllocCPUS   TotalCPU ExitCode
------------ ---------- ---------- ---------- ---------- ---------- ---------- --------
6417753      fastqc_de+  COMPLETED   00:00:05                     4  00:10.350      0:0
6417753.bat+      batch  COMPLETED   00:00:05       3416K         4  00:10.349      0:0
6417753.ext+     extern  COMPLETED   00:00:05        920K         4   00:00:00      0:0
```

# Scripting on the cluster — Slurm: cancel a job

**When do we cancel a job?**

- A job does not meet the requirements and remains in the **PD (pending)** state.
- I made a mistake when submitting or configuring the job and want to cancel it before it continues.
- I realize that I allocated more resources than necessary.
- Something unexpected happened and the job appears to be stuck.



# Scripting on the cluster — Slurm: cancel a job

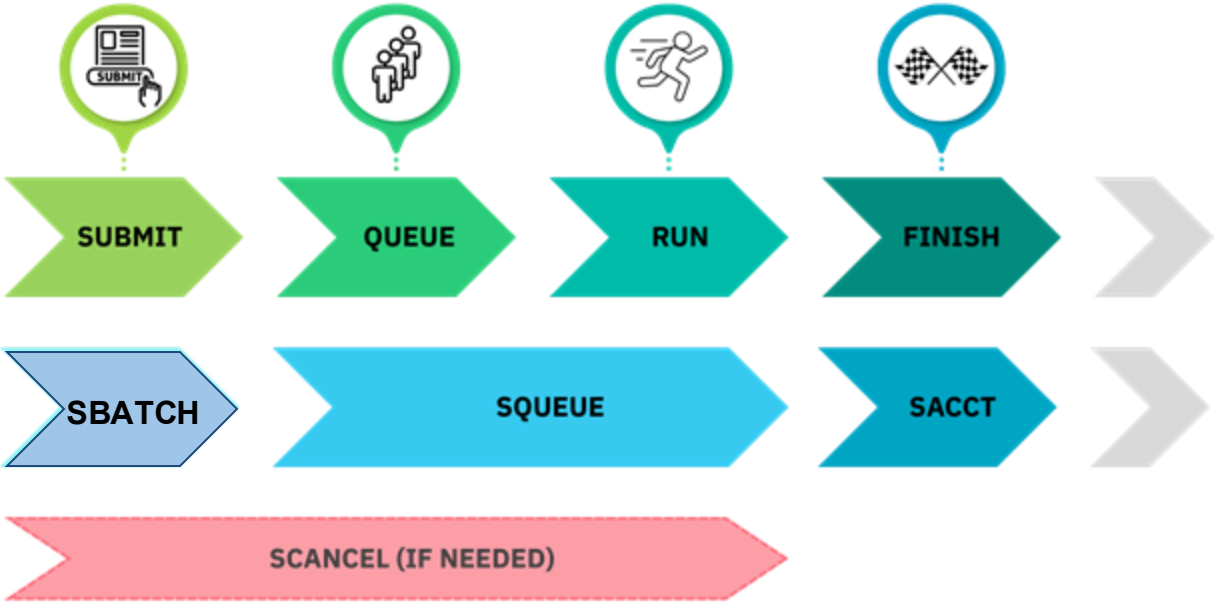
## When do we cancel a job?

```
squeue -u minombre
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASC
           6417754    shared fastqc_de minombre  R      12:40      1 node01
           6417910    shared fastqc_de minombre  PD      0:00      1 (Priority)
           6417911    shared fastqc_de minombre  PD      0:00      1 (Priority)

scancel --state=PENDING --user=minombre

squeue -u minombre
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASC
           6417754    shared fastqc_de minombre  R      12:45      1 node01
```

# Monitoring and job states



# Scripting on the cluster — Slurm: sbatch




**Bigger asks = longer wait, not faster.**

# Scripting on the cluster — Slurm: Redirecting Job Output

## Once the job/s have finished → Output files

- **Standard output**: Standard outputs of your program.
  For example, if you used: **--output=slurm-%j.out**



**slurm-12345.out**

- **Standard error**: Potential errors you face during execution appear here.
  For example, if you used: **--error=slurm-%j.err**



**slurm-12345.err**

# Scripting on the cluster — Take-home message

## Good Practices:

- ✅ Descriptive **--job-name**; comment your script.
- ❌ Never run heavy jobs on the login node !!!! Use **sbatch/srun**.
- ❌ Do not request more resources than need!!!!
- ✅ Test small first, then scale.
- ✅ Version your scripts (Git).



# Thank you for your attention

Questions?

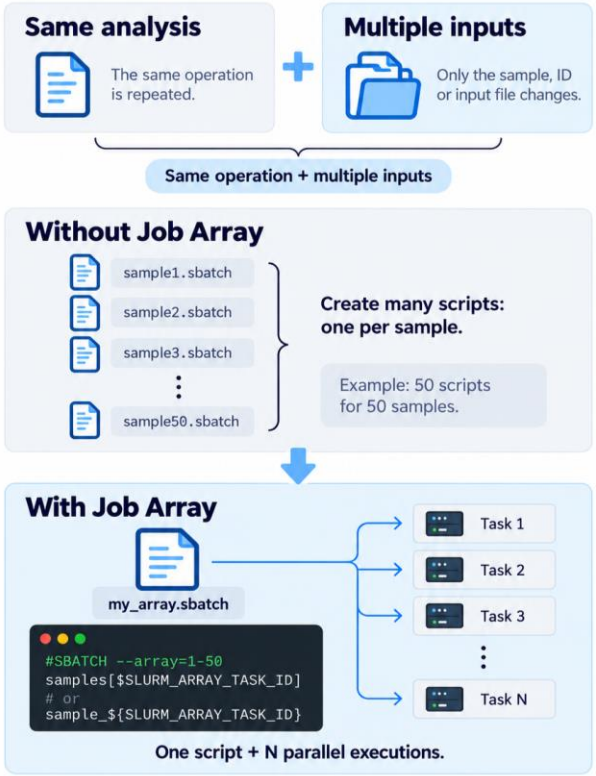
# JobArrays

## (Embarrassingly parallel)

# Scripting on the cluster — Slurm: job arrays

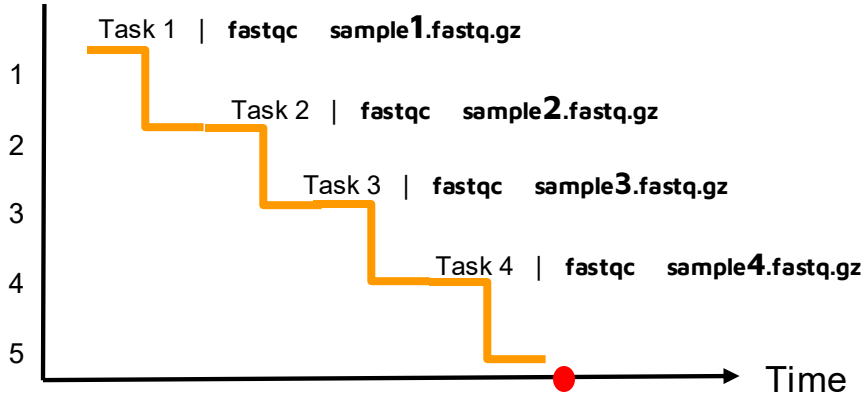
## What are JobArrays and what are they used for?

- Sbatch **script designed to launch many jobs**
  - Same operation / multiple inputs
- Instead of creating 50 scripts for 50 samples, you create one script and tell Slurm to run it N times → **In Parallel**
- If your shell for loop only changes a filename/ID, make it a job array.



# Scripting on the cluster — Slurm: job arrays

## For Loop




## JobArray



# Scripting on the cluster — Slurm: job arrays

## Create a JobArray script?

- As simple as using the `--array` inside the sbatch script
- **Plus Slurm env variables**

https://docs.hpc.shef.ac.uk/en/latest/referenceinfo/scheduler/SLURM/SLURM-environment-variables.html#gsc.tab=0

```
#!/bin/bash
#SBATCH --chdir=/path/to/project
#SBATCH --job-name=fastqc_array
#SBATCH --partition=short_idx
#SBATCH --array=1-20
#SBATCH --cpus-per-task=1
#SBATCH --mem=5G
#SBATCH --time=00:15:00
#SBATCH --output=fastqc_%A_%a.out # %A: array JobID, %a: task index
#SBATCH --error=fastqc_%A_%a.err

module load fastqc/0.12.1

mkdir fastqc_results
fastqc -o fastqc_results muestra_${SLURM_ARRAY_TASK_ID}.fq.gz
```

# Scripting on the cluster — Slurm: job arrays

## Monitoring Arrays

```
$ sbatch fastqc_array_20.sbatch
Job Submitted
$ squeue --me
             JOBID PARTITION     NAME      USER ST   TIME  NODES NODELIST(REASON)
          78901_1   short_idx fastqc_array   dani  R   0:41      1 ideafix02
          78901_2   short_idx fastqc_array   dani  R   0:39      1 ideafix03
          78901_3   short_idx fastqc_array   dani  R   0:36      1 ideafix04
          78901_4   short_idx fastqc_array   dani PD   0:00      1 (Resources)
          78901_5   short_idx fastqc_array   dani PD   0:00      1 (Resources)
          78901_6   short_idx fastqc_array   dani PD   0:00      1 (Resources)
          78901_7   short_idx fastqc_array   dani PD   0:00      1 (Priority)
          78901_8   short_idx fastqc_array   dani PD   0:00      1 (Priority)
          78901_9   short_idx fastqc_array   dani PD   0:00      1 (Priority)
         78901_10   short_idx fastqc_array   dani PD   0:00      1 (Priority)
```

# Scripting on the cluster — Slurm: job arrays

**Get sample's file name using array index?**

```
#!/bin/bash
#SBATCH --job-name=fastqc_from_ids
#SBATCH --partition=short_idx
#SBATCH --array="1-20%5"
#SBATCH --cpus-per-task=1
#SBATCH --mem=6G
#SBATCH --output=logs/%x-%A_%a.out
#SBATCH --error=logs/%x-%A_%a.err
module load FastQC/0.11.9-Java-11

IDS_FILE="../samples_id.txt"
READS_DIR="../00-reads"

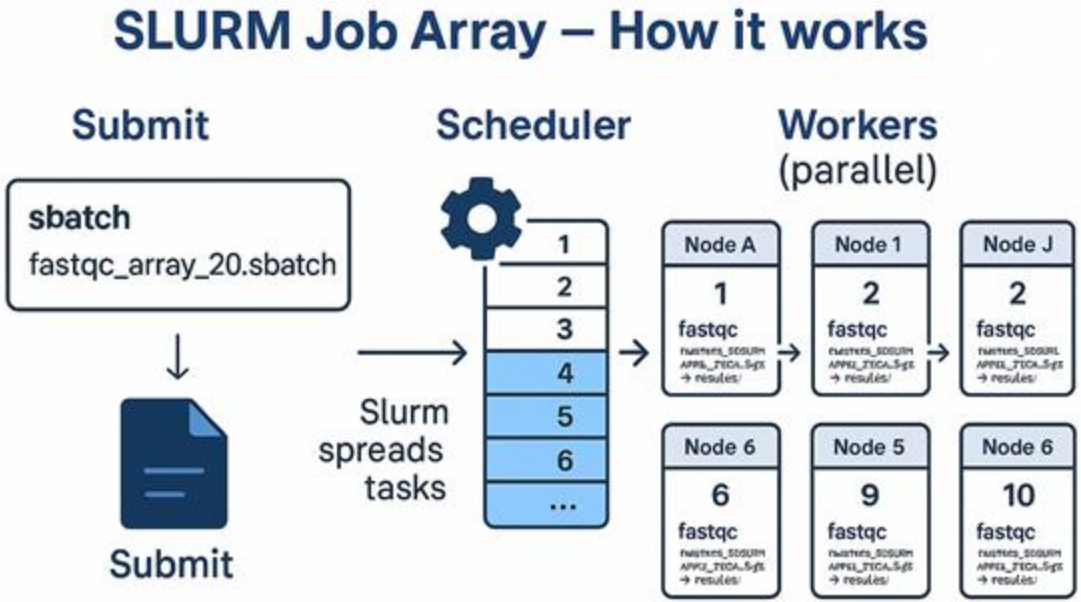
RESULTS_ROOT="03-fastqc-array-results"
mkdir -p "$RESULTS_ROOT"
      You, 2 minutes ago • Uncommitted changes
SAMPLE=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$IDS_FILE")

[[ -n "$SAMPLE" ]] || { echo "[ERROR] No sample for task ${SLURM_ARRAY_TASK_ID}" >&2; exit 2; }
R1="${READS_DIR}/${SAMPLE}_R1.fastq.gz"
R2="${READS_DIR}/${SAMPLE}_R2.fastq.gz"
[[ -r "$R1" && -r "$R2" ]] || { echo "[ERROR] Missing reads for ${SAMPLE}" >&2; exit 3; }

OUTDIR="${RESULTS_ROOT}/fastqc_from_ids_${SLURM_ARRAY_JOB_ID}_${SLURM_ARRAY_TASK_ID}_${SAMPLE}"
mkdir -p "$OUTDIR"

fastqc -o "$OUTDIR" "$R1" "$R2"
echo "[INFO] Sample=${SAMPLE} R1=${R1} R2=${R2}"
```

# Scripting on the cluster — Slurm: job arrays



# Thank you for your attention

Questions?

# Index

1. Scripting on the cluster — sbatch & job arrays
2. **Parallel computing on the cluster — OpenMP vs MPI (when to use each)**
3. Nextflow pipelines in HPC  - Deploy a Nextflow DSL2 pipeline in HPC
4. Wrap-up & Q&A

# OpenMP v.s. MPI

(Open Multi-Processing) (Message Passing Interface

# Parallelization on our cluster — Why parallelize?

**Why Parallelize?**

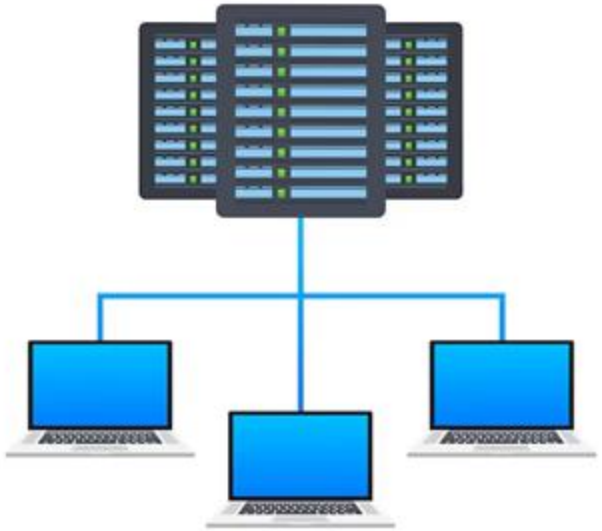
To reduce time-to-results by using multiple computing resources simultaneously.

**Parallel programming can:**

- Get your analysis done in minutes instead of hours or days.
- Maximize the utilization of available cluster resources.
- Enable massive, computationally demanding workflows (e.g., population-scale genomics)

**Main Parallel Models in HPC**

- OpenMP — Shared-Memory Parallelism
  Multiple threads (~CPUs) work together inside a single node, sharing a common pool of RAM.
- MPI — Distributed-Memory Parallelism
  Multiple independent processes work across multiple separate nodes, communicating data over the high-speed network.



**In short:**
**OpenMP → one node, many threads**
**MPI → multiple processes, potentially across many nodes**

# Parallelization on our cluster — OpenMP

## OpenMP Essentials

- **Intra-Node Parallelism:**

multiple computational workers (**threads**) inside a single process running on one node.

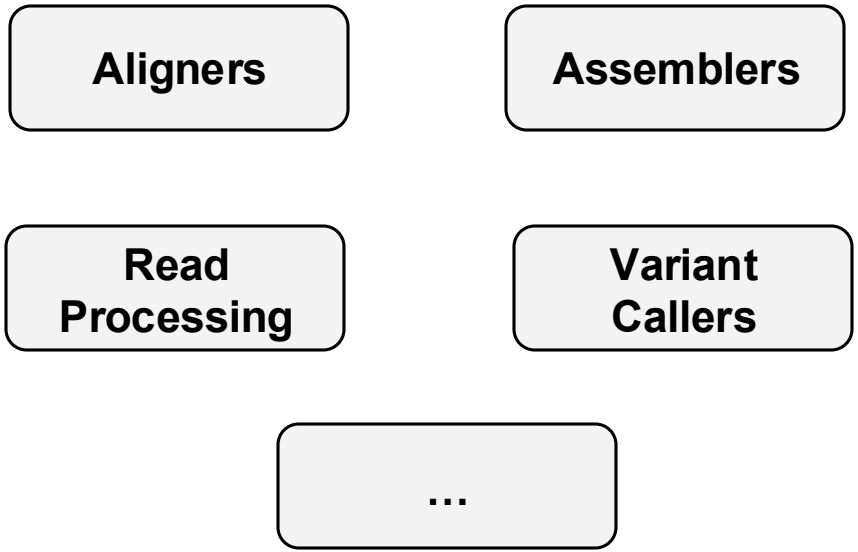
- **1 Thread ≈ 1 CPU:**

The program dynamically splits its workload among the available CPU cores of that machine.

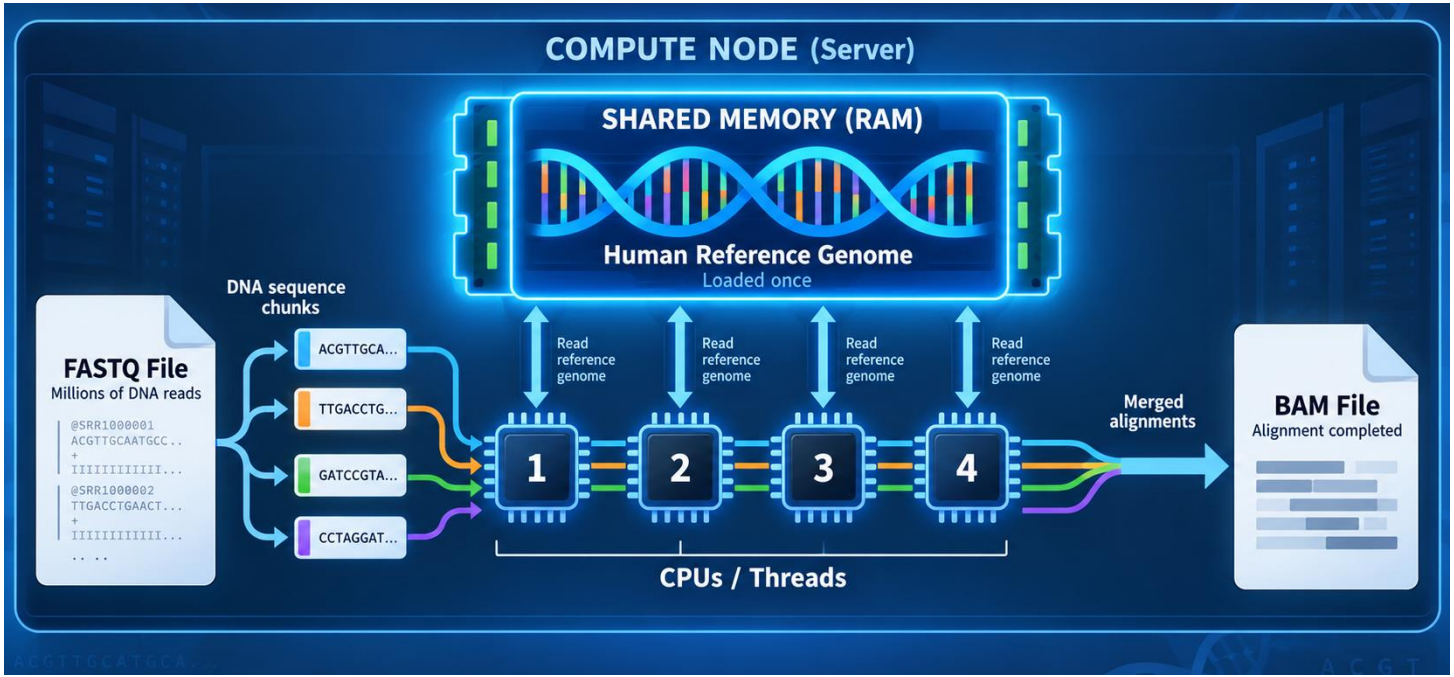
- **Shared RAM:**

Every thread can directly read from and write to the same central memory space.

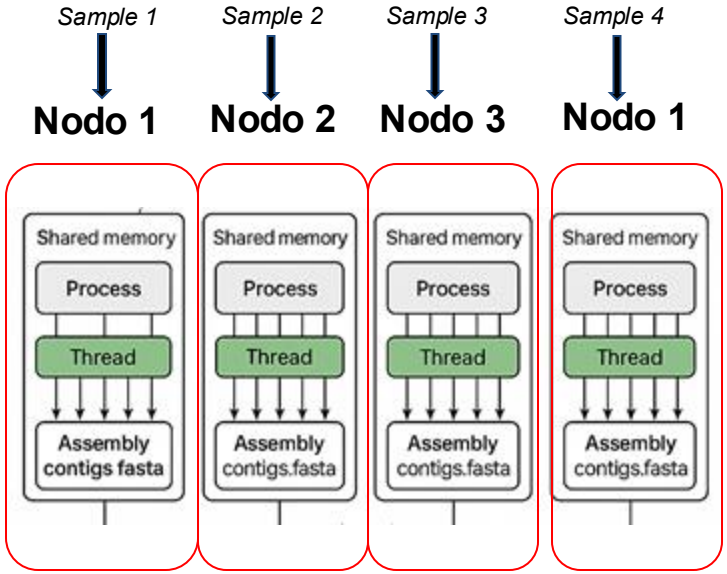
**Most bioinformatics tools use the OpenMP model**



# Parallelization on our cluster — OpenMP



# Parallelization on our cluster — OpenMP



# Parallelization on our cluster — OpenMP

```
#!/bin/bash
#SBATCH --job-name=spades_openmp
#SBATCH --partition=short_idx
#SBATCH --cpus-per-task=16
#SBATCH --mem=32G
#SBATCH --time=02:00:00
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err

module load SPAdes/3.15.2-GCC-10.2.0

mkdir -p 04-openmp-spades-results

R1=../00-reads/ERR2261314_R1.fastq.gz
R2=../00-reads/ERR2261314_R2.fastq.gz

spades.py -1 "$R1" -2 "$R2" -o 04-openmp-spades-results/spades_sample01 \
    --threads "$SLURM_CPUS_PER_TASK" \
    --mem $SLURM_MEM_PER_NODE
```

# Parallelization on our cluster — MPI

## MPI Essentials

- It is designed to run an application using **multiple separate processes**, possibly on **different nodes** of a cluster
- When: multinode scaling & very large memory needs.
- Shares resources by passing messages over the cluster's network → network issues can stop the job

**Only a few bioinformatics tools use the MPI model**

**RAxML**

**IQ-Tree**

# Parallelization on our cluster — MPI

## MPI Essentials

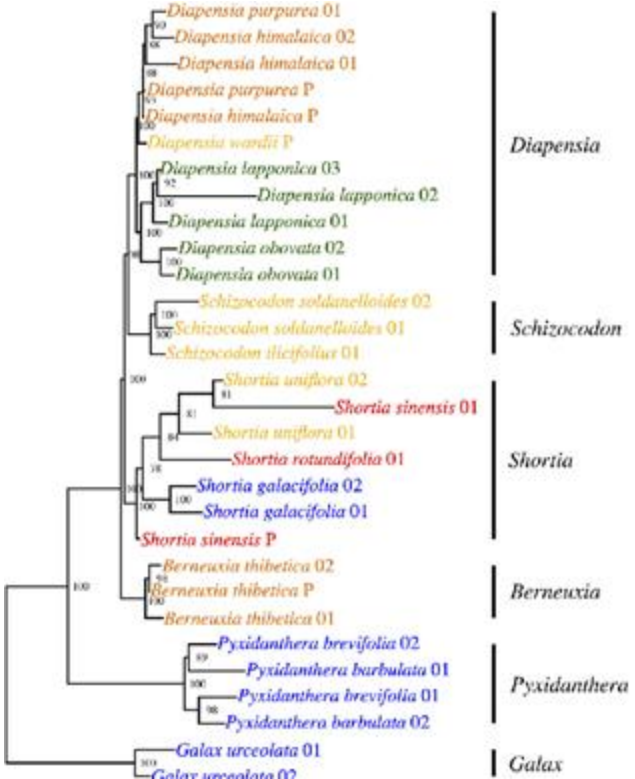
- It is designed to run an application using **multiple separate processes**, possibly on **different nodes** of a cluster
- When: multinode scaling & very large memory needs.
- Shares resources by passing messages over the cluster's network → network issues can stop the job

**Otras áreas**

- **Modelos de propagación de enfermedades**
- **Reconstrucción de imágenes 3D**
- **Modelos climáticos y de contaminación**
- **Sincronización de modelos ML (gradient descent trees)**

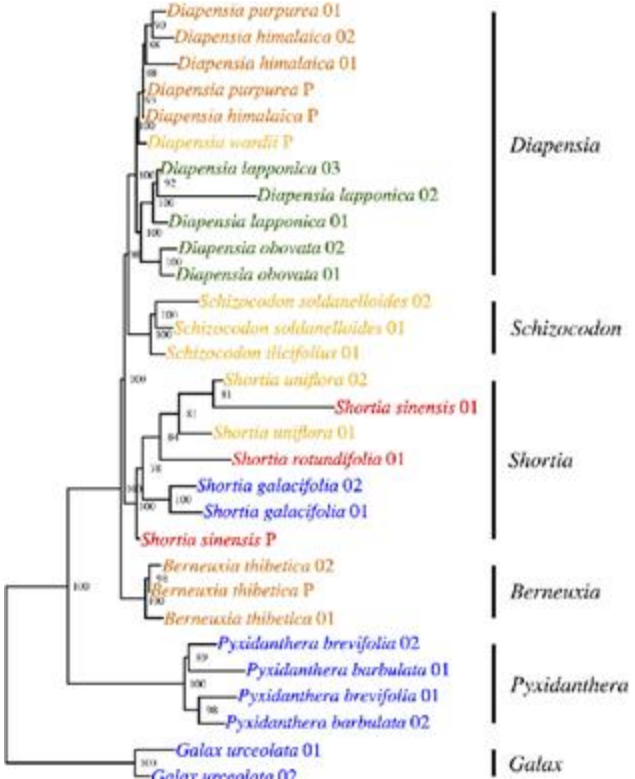
# Ejemplo de **MPI** con herramienta bioinfo **RAxML**

- Imagina un alineamiento de 100.000 secuencias.
- No cabe en memoria de un solo nodo.
- Necesitas usar **varios nodos**, cada uno procesa una parte del árbol y comparten información vía mensajes.



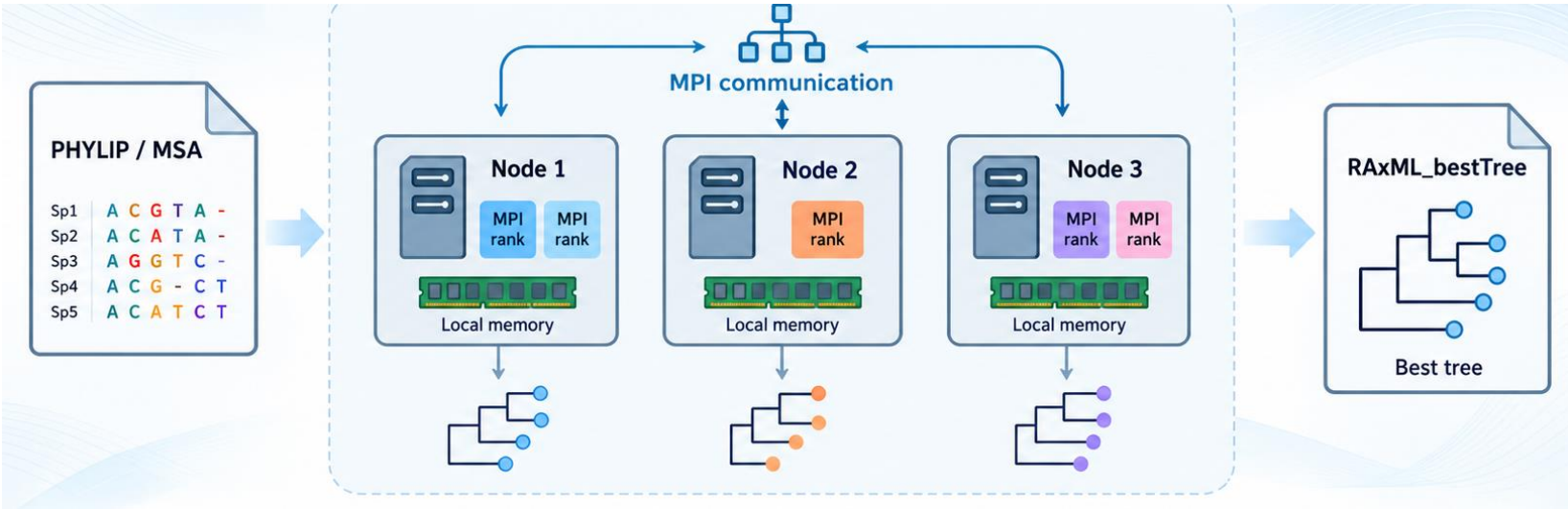
# Ejemplo de **MPI** con herramienta bioinfo **RAxML**

- Imagina un alineamiento de 100.000 secuencias.
- No cabe en memoria de un solo nodo.
- Necesitas usar **varios nodos**, cada uno procesa una parte del árbol y comparten información vía mensajes.



# Ejemplo de **MPI** con herramienta bioinfo **RAxML**

MPI ranks communicate by exchanging messages; across nodes, those messages travel through the cluster network.



# Ejemplo de **MPI** con herramienta bioinfo **RAxML**

```
#!/bin/bash
#SBATCH --job-name=raxml_mpi
#SBATCH --partition=short_idx
#SBATCH --nodes=2                  # <-- nº de nodos
#SBATCH --ntasks=8                 # total procesos MPI
#SBATCH --ntasks-per-node=4        # <-- nº procesos MPI por nodo
#SBATCH --mem=8G
#SBATCH --time=00:30:00
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err

module load RAxML/8.2.12-gompi-2020a-hybrid-avx2  # el módulo puede traer varios binarios

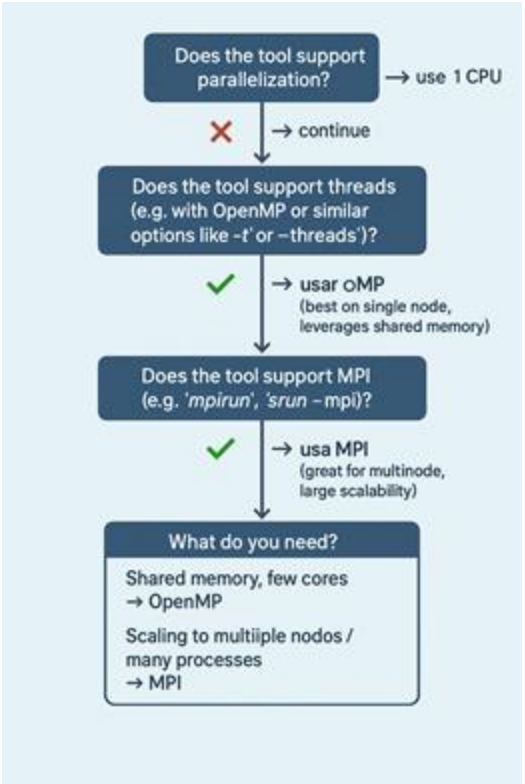
mpirun -np "$SLURM_NTASKS" raxmlHPC \
  -s data/datos.phy \
  -m GTRGAMMA \
  -p 12345 \
  -# 20 \
  -n "$RUNNAME" \
  -w "$RESULTS_DIR"
```

# Parallelization on our cluster — OpenMP vs MPI

| KEY DIFFERENCES | | | | |
|---|---|---|---|---|
| **Technology** | **Parallelization level** | **Communication between processes** | **Typical use case** | **Usage (sbatch script)** |
| **OpenMP** | Within **one node** | Shared memory: all threads access the same RAM | Align 200 million reads on one node using all its cores | **--cpus-per-task (threads)**<br>**--mem (Ram for the whole process)** |
| **MPI** | Across **multiple nodes** | Distributed memory: each node has its own RAM; communication by network | Build a very large phylogenetic tree by splitting the work over 4 nodes | **--nodes**<br>**--ntasks**<br>**--ntasks-per-node** |

# Parallelization on our cluster — OpenMP vs MPI

## When to use OpenMP vs MPI?



# Thank you for your attention

Questions?



# HANDS-ON